AI Features Documentation

```
# AI Features Documentation

## 1. Executive Overview
- The product has two AI tracks:
- Conversation AI suite (triage, memory/tasks, draft writing, summary report + PDF, telemetry).
- Legacy/adjacent AI flows (generic ChatGPT helper and omnichannel lead-intent classification job).
- Customer-facing AI:
- Conversation panel in Conversations (triage, memory, draft, report export).
- Floating ChatGPT modal (global helper).
- Internal/supporting AI:
- Provider client, prompt factories, schema/repair guards, telemetry logs, cost estimation, superadmin AI usage
dashboard.
- Overall maturity assessment:
- Conversation AI core is operational and integrated (UI + endpoints + persistence + tests).
- Report processing is currently sync-in-controller despite queued-job scaffolding.
- Feedback endpoint/logging and some docs are ahead of implementation.
- Omnichannel classification is partially implemented across channels (Facebook/WordPress wired, WhatsApp TODO).

## 2. AI Feature Inventory Table

| Feature | Purpose | Status | User-Facing or Internal | Main Entry Point | Main Files | Key Dependency | Main
Risk |
|---|---|---|---|---|---|---|---|
| Conversation AI Triage | Classify thread state/risk/actionability and strategic next action | Live | User-Facing
| `GET/POST /ai/triage/{thread}` | `ConversationAiTriageController`, `ConversationAiTriageService`, `TriageSkill`
| OpenAI Responses API | Complex rule engine may drift from UI expectations |
| Conversation AI Memory + AI Tasks | Relationship memory and task checklist with completion updates | Live |
User-Facing | `GET /ai/memory/{id}`, `PATCH /ai/tasks/{task}` | `ConversationAiMemoryController`,
`ConversationAiMemoryService`, `MemorySkill` | Triage history + OpenAI | Memory quality depends on linked
entity/thread context |
| Conversation AI Draft Assistant | Generate triage-aware reply drafts with guardrails | Live | User-Facing |
`POST /ai/draft` | `ConversationAiDraftController`, `ConversationAiDraftService`, `DraftSkill` | OpenAI + triage
snapshot | Blocking logic can surprise users without clear explainability |
| AI Summary Report + PDF | Build executive summary and downloadable PDF | Live (sync), Partially Implemented
(async path) | User-Facing | `POST /ai/reports/generate`, `GET /ai/reports/{job}/download` |
`ConversationAiReportController`, `ConversationAiReportService`, `ReportSkill` | Context builder + PDF (DomPDF) |
Docs/architecture imply async, code runs sync |
| AI Telemetry + Cost + Usage Insights | Track AI usage/cost/success and visualize in superadmin dashboard |
Internal Only | Internal | Service calls + superadmin dashboard render | `ConversationAiTelemetryService`,
`AiUsageCostCalculator`, `DashboardController` | `ai_usage_logs` | Cost model map omits `gpt-5.4-mini` explicit
pricing |
| Legacy ChatGPT Content Generator | Generic prompt-based content generation modal | Live | User-Facing | `POST
/api/chatgpt/generate` | `ChatGptController`, `ChatGptModal`, `FloatingChatGpt` | OpenAI chat completions |
Exception messages may leak internals to client |
| Omnichannel Lead Intent Classification | Auto-classify inbound lead events and update lead AI fields | Partially
Implemented | Internal Only | `LeadEventTrackerService` job dispatch | `ClassifyLeadIntentJob`,
`LeadEventTrackerService` | OpenAI + webhook ingestion | No explicit confidence threshold gating; WhatsApp not
wired |
| AI Feedback Logging Scaffold | Intended feedback loop endpoint/log storage | In Progress | Internal Only | No
active route/controller | AI feedback migration exists; docs reference missing classes | N/A | Stakeholder
confusion: documented but not implemented |
| Conversation AI Availability/Gating Controls | Enable/disable AI and configure provider/model/timeout | Disabled
/ Gated | Internal | `POST settings/chatgpt` | `SystemSettingsController`, `ConversationAiConfigService`,
`ConversationAiRules` | Superadmin settings | Conversation endpoints are not plan-feature-gated directly |

## 3. Detailed Feature Documentation
```

Feature: Conversation AI Triage

Purpose
- Classifies each conversation thread into commercial state, relationship health, actionability, and success probability.
- Produces one strategic action recommendation for next step.
- Business value: improves response prioritization and reduces subjective triage variance.

Status
- Live.
- Evidence: active routes, controller/service/provider wiring, persistence model + migration, UI cards/drawer, tests in `tests/Feature/AI`.

Where It Appears
- UI: Conversations AI drawer/panel.
- Routes: `GET /ai/triage/{thread}`, `POST /ai/triage/{thread}/refresh`.
- Internal trigger: auto-refresh when new thread activity is newer than analyzed timestamp (+5s buffer).

Primary Users
- Sales/support staff in conversation inbox.
- Team leads using triage summary for prioritization.

Feature Flow
1. UI calls triage endpoint for selected thread.
2. Controller enforces tenant ownership and AI availability.
3. Service loads existing triage; refreshes if stale/new activity.
4. Skill builds prompt and calls provider; applies parse/policy/state-transition guards.
5. Result stored in `ai_triage_results` via `updateOrCreate`.
6. Response returned to UI with mapped `suggested_status`.

Inputs
- Email thread subject/snippet/participants.
- Recent messages (up to 8 in prompt factory).
- Prior triage snapshot (for transition enforcement).
- Required: valid thread ID in tenant scope.

Outputs
- `intent`, `intent_confidence`, `priority`, `thread_state`, `relationship_health`, `actionability`, `success_probability`, `behavioral_pulse`, `summary`, `strategic_action`.
- Stored in DB and shown in AI panel badges/cards.
- Telemetry row logged on success/failure.

Thresholds / Rules / Conditions
- Availability gate: `enabled && api_key`.
- Route-level throttle: `30 requests/minute`.
- Auto-refresh stale buffer: `5` seconds.
- Score clamps and caps:
- percent values `0..100`.
- `closed_lost <=5`.
- `objection <=55`.
- `misaligned <=30`.
- reopened calibrated `25..45`.
- fallback caps `intent_confidence<=60`, `success_probability<=50`.
- Urgency downgrade unless intent in `sales|billing`.
- Recommendation policy must pass module prefix + semantic checks.
- Tenancy checks via `created_by`.
- No explicit threshold found:
- no hard token limit in endpoint code.

Main Files
- `routes/web.php`

- `app/Http/Controllers/AI/ConversationAiTriageController.php`
- `app/Services/AI/ConversationAiTriageService.php`
- `app/Services/AI/Skills/TriageSkill.php`
- `app/Services/AI/Prompts/TriagePromptFactory.php`
- `app/Models/AiTriageResult.php`

Dependencies
- OpenAI provider client.
- Prompt factory + JSON schema contract.
- Telemetry service and `ai_usage_logs`.

Telemetry / Logging
- Success/failure logged via `ConversationAiTelemetryService` with token counts and repair/fallback metadata.

Risks / Caveats
- High rule complexity can create edge-case behavior difficult to explain.

Current Gaps
- No explicit per-feature latency SLO/threshold in code.

Presentation Summary
- Live production triage with deterministic post-processing guards.
- Strong tenancy and availability fail-safe controls.
- Business-safe constraints prevent optimistic misclassification in terminal states.
- Observability exists through feature-level usage/failure logs.

Feature: Conversation AI Memory + AI Tasks

Purpose
- Maintains relationship memory summary and actionable checklist tied to contact/lead.
- Business value: continuity across thread history and structured follow-up tracking.

Status
- Live.

Where It Appears
- UI: AI Memory card in conversation panel.
- Endpoints: `GET /ai/memory/{id}?entity_type=contact|lead`, `PATCH /ai/tasks/{task}`.

Primary Users
- Sales/account teams handling ongoing relationships.

Feature Flow
1. UI resolves linked contact/lead from thread.
2. Memory endpoint fetches latest summary by entity + tenant.
3. Service auto-refreshes if newer entity/thread activity detected (+5s).
4. Skill generates/repairs memory; stores new summary row.
5. Tasks loaded and shown; task toggle updates completion via PATCH.
6. Telemetry success/failure logged.

Inputs
- Entity (`contact` or `lead`) in tenant.
- Recent linked thread IDs (up to 5) and related triage context.
- Required validation: `is_completed` boolean for task updates.

Outputs
- Relationship summary, relationship strength, memory points, tasks.
- New summary persisted; task completion state persisted.

Thresholds / Rules / Conditions
- Auto-refresh stale buffer: `5` seconds.

- Triage-context threads considered: up to `5`.
- Relationship strength enum: `weak|moderate|strong`.
- Fallback memory applied on parse/policy failure.
- Reconciliation clamps optimism in `closed_lost`, `damaged`, `stalled`, `reopened`.
- Task update requires AI availability.
- No explicit threshold found:
- no max task count or memory length in service layer.

Main Files
- `app/Http/Controllers/AI/ConversationAiMemoryController.php`
- `app/Http/Controllers/AI/ConversationAiTasksController.php`
- `app/Services/AI/ConversationAiMemoryService.php`
- `app/Services/AI/Skills/MemorySkill.php`
- `app/Models/AiMemorySummary.php`
- `app/Models/AiTask.php`

Dependencies
- Triage outputs (context enrichment).
- Provider + memory prompt factory.

Telemetry / Logging
- `memory_show` success/failure records in `ai_usage_logs`.

Risks / Caveats
- Memory freshness depends on correct entity linkage to threads.

Current Gaps
- No dedicated task SLA/priority engine beyond stored priority text.

Presentation Summary
- Memory and task assistance are live and persisted.
- Uses recent triage states for continuity and risk-aware memory.
- Auto-refresh keeps summaries current after new activity.
- Strong fallback behavior avoids empty responses.

Feature: Conversation AI Draft Assistant

Purpose
- Generates suggested email replies using triage-aware guardrails.
- Business value: faster response drafting with risk-aware communication constraints.

Status
- Live.

Where It Appears
- UI: reply assistant card + editor popover in Conversations.
- Endpoint: `POST /ai/draft`.

Primary Users
- Conversation owners and assigned reps.

Feature Flow
1. User triggers draft generation with prompt/tone.
2. Controller validates input and tenant thread ownership.
3. Service loads latest triage for the thread.
4. Skill applies pre-call triage guards, then AI generation, then policy checks/repair.
5. Draft stored in `ai_draft_runs` when not hard-blocked.
6. Draft returned; UI inserts into editor on user action.

Inputs
- `threadId` (required), `prompt` (required), `tone` (`max:50`).

```
- Latest triage result for state-aware constraints.

**Outputs**
- Subject/body draft + generated timestamp.
- DB row in `ai_draft_runs` (except hard-blocked cases).

**Thresholds / Rules / Conditions**
- Hard-block `do_not_pursue`.
- Hard-block `closed_lost` unless recovery keyword appears.
- Subject trim max `140`.
- Body must start with `<p>`.
- Max one `?` in body.
- Misaligned blocks scheduling language.
- Closed-lost/damaged blocks aggressive sales language.
- 422 block response includes `blocked=true` and reason.

**Main Files**
- `app/Http/Controllers/AI/ConversationAiDraftController.php`
- `app/Services/AI/ConversationAiDraftService.php`
- `app/Services/AI/Skills/DraftSkill.php`
- `resources/js/pages/conversations/components/EditorAiAssistant.tsx`
- `resources/js/pages/conversations/components/AiReplyAssistantCard.tsx`

**Dependencies**
- Triage snapshot.
- OpenAI provider via strict JSON schema.

**Telemetry / Logging**
- `draft` success/failure usage logs with tone, prompt_version, token counts.

**Risks / Caveats**
- Strict guards can block generation in states users may not expect.

**Current Gaps**
- Policies are hardcoded, not settings-driven.

**Presentation Summary**
- Draft assistant is live with strong risk-aware controls.
- Hard blocks prevent unsafe outreach in terminal states.
- Outputs are structured and auditable.
- Fast UX path: generate -> preview -> insert.

### Feature: AI Summary Report + PDF Download

**Purpose**
- Produces executive-style account/conversation summary and downloadable PDF.
- Business value: stakeholder-ready summaries for reviews and decision meetings.

**Status**
- Live (sync processing path), with Partially Implemented async architecture.

**Where It Appears**
- UI: â€œDownload Summary Reportâ€ in AI Triage card.
- Endpoints: `/ai/reports/options/{thread}`, `/ai/reports/generate`, `/ai/reports/{job}`,
`/ai/reports/{job}/download`.
- Background job class exists but is not currently dispatched by controller.

**Primary Users**
- Account managers, leadership reviewers, customer-facing owners.

**Feature Flow**
```

1. UI loads report scope options.
2. User posts generate request with scope/opportunity.
3. Controller validates scope and linked contact/opportunity eligibility.
4. Service creates `ai_report_jobs` row (`status=queued`) and builds context payload.
5. Controller immediately calls `process()` synchronously.
6. Skill generates and normalizes report sections; job status updates to `completed|fallback|failed`.
7. Download endpoint renders PDF via Blade template + formatter.

Inputs
- Required: `threadId`.
- Optional: `scope`, `contactId`, `opportunityId`.
- `opportunityId` required when `scope=specific-opportunity`.
- Context from thread, CRM entities, activity streams, triage snapshot.

Outputs
- Job record with result/context/metadata payloads.
- PDF file `AI-Summary-Report-{job}.pdf`.
- Telemetry entry `report_generate`.

Thresholds / Rules / Conditions
- Scope enum: `overall|leads-only|all-opps|specific-opportunity`.
- `opportunityId` required_if specific-opportunity.
- Selected contact/opportunity must be linked to context.
- Activity stream cap: `250`.
- Opportunity list in context limited to top `20`.
- Download returns `409 report_result_unavailable` when result absent.
- Status transitions: `queued -> completed|fallback|failed`.
- Queue behavior:
- Job class exists, but controller executes sync path inline.

Main Files
- `app/Http/Controllers/AI/ConversationAiReportController.php`
- `app/Services/AI/ConversationAiReportService.php`
- `app/Services/AI/ConversationAiReportContextBuilder.php`
- `app/Services/AI/Skills/ReportSkill.php`
- `app/Services/AI/Reports/ReportTemplateFormatter.php`
- `resources/views/reports/ai_summary_pdf.blade.php`
- `app/Jobs/AI/GenerateConversationAiReportJob.php`

Dependencies
- OpenAI provider and report prompt/schema.
- Context builder + activity digest builders.
- DomPDF.

Telemetry / Logging
- `report_generate` success/failure with token metrics.
- `ReportSkill` includes debug logs with prompt/response details.

Risks / Caveats
- Privacy risk: full prompt/raw response debug logging.
- Docs mismatch: docs describe async queue/feedback endpoint; runtime differs.

Current Gaps
- Async queue path scaffolded but not active.
- Feedback workflow documented but missing runtime implementation.

Presentation Summary
- Report + PDF export is live for users.
- Context quality and section normalization are strong.
- Runtime currently sync (not fully queue-driven).
- Validation prevents out-of-context entity selection.

```
### Feature: AI Telemetry + Usage Insights

**Purpose**
- Capture usage/failure/cost signals and expose dashboard metrics.

**Status**
- Internal Only.

**Where It Appears**
- Internal service calls from AI controllers/services.
- Superadmin dashboard insights panel.

**Primary Users**
- Superadmin, operations, finance stakeholders.

**Feature Flow**
1. AI features call telemetry service for success/failure.
2. Usage logs stored with tokens/model/metadata.
3. Dashboard aggregates 30-day totals, success rate, trends, model distribution, top companies.
4. Superadmin UI visualizes insights.

**Inputs**
- Feature name, model, tokens, metadata.

**Outputs**
- `ai_usage_logs` rows.
- Dashboard KPIs and charts.

**Thresholds / Rules / Conditions**
- 30-day aggregation window.
- Success rate computed from metadata status (`JSON` query with `LIKE` fallback).
- No explicit alert thresholds in app code.

**Main Files**
- `app/Services/AI/ConversationAiTelemetryService.php`
- `app/Services/AI/AiUsageCostCalculator.php`
- `app/Http/Controllers/DashboardController.php`
- `resources/js/components/dashboard/ai-usage-insights.tsx`

**Dependencies**
- `ai_usage_logs` persistence.

**Telemetry / Logging**
- This is the telemetry layer.

**Risks / Caveats**
- Cost estimate map lacks explicit `gpt-5.4-mini` pricing entry.

**Current Gaps**
- No in-app alerting thresholds or notification logic.

**Presentation Summary**
- Internal observability is operational and dashboarded.
- Success/failure and token tracking are robust.
- Cost estimates are heuristic, not billing-grade.

### Feature: Legacy ChatGPT Content Generator

**Purpose**
- General prompt-based text generation helper.
```

```
**Status**
- Live.

**Where It Appears**
- Global floating button/modal.
- Endpoint: `POST /api/chatgpt/generate`.

**Primary Users**
- Users with AI plan entitlement and visible UI access.

**Feature Flow**
1. User enters prompt/options in modal.
2. Frontend posts to endpoint.
3. Controller validates and calls OpenAI chat completions.
4. Returns generated text.

**Inputs**
- `prompt`, `language`, `creativity`, `num_results`, `max_length`.

**Outputs**
- Generated text.

**Thresholds / Rules / Conditions**
- `prompt max:1000`.
- `num_results:1..5`.
- `max_length:1..500`.
- Language + creativity whitelists.
- Temperature mapping (`0.3/0.7/0.9`).
- UI plan gate uses `ai_integration`.

**Main Files**
- `app/Http/Controllers/ChatGptController.php`
- `resources/js/components/FloatingChatGpt.tsx`
- `resources/js/components/chatgpt/ChatGptModal.tsx`

**Dependencies**
- OpenAI PHP client + legacy settings (`chatgptKey`, `chatgptModel`).

**Telemetry / Logging**
- No dedicated AI usage telemetry in this path.

**Risks / Caveats**
- Raw exception message returned to client on failure.

**Current Gaps**
- No usage/cost logging parity with Conversation AI.

**Presentation Summary**
- Legacy assistant remains active.
- Simpler path than Conversation AI.
- Useful but less governed/observable.

### Feature: Omnichannel Lead Intent Classification

**Purpose**
- Asynchronously classify inbound lead intent/stage and update lead AI fields.

**Status**
- Partially Implemented.
```

```
**Where It Appears**
- Triggered by `LeadEventTrackerService::recordInboundEvent`.
- Wired through Facebook and WordPress webhook paths.
- WhatsApp path still TODO.

**Primary Users**
- Internal automation supporting sales ops.

**Feature Flow**
1. Webhook ingests inbound event.
2. Tracker creates/updates contact + lead and stores lead event.
3. Tracker dispatches `ClassifyLeadIntentJob`.
4. Job calls OpenAI and parses JSON.
5. Stores `ai_classification_results` and updates lead AI fields.

**Inputs**
- Lead event summary text and current opportunity stage list.

**Outputs**
- Classification row and updated lead AI fields.

**Thresholds / Rules / Conditions**
- Skips if no event summary text.
- Skips with warning if API key missing.
- Temperature fixed at `0.2`.
- No explicit confidence cutoff before writing lead fields.

**Main Files**
- `app/Services/Omnichannel/LeadEventTrackerService.php`
- `app/Jobs/ClassifyLeadIntentJob.php`
- `app/Models/AiClassificationResult.php`
- `app/Http/Controllers/Webhooks/FacebookWebhookController.php`
- `app/Http/Controllers/Webhooks/WordPressWebhookController.php`
- `app/Http/Controllers/Webhooks/WhatsAppWebhookController.php`

**Dependencies**
- Webhook ingestion + OpenAI client.

**Telemetry / Logging**
- Warning/error logging only; no centralized usage telemetry.

**Risks / Caveats**
- Channel coverage incomplete (WhatsApp TODO).
- No confidence-threshold gating.

**Current Gaps**
- Missing policy threshold for automatic stage updates.

**Presentation Summary**
- Real async AI classification exists and is wired in production webhook flows.
- Facebook and WordPress are integrated.
- WhatsApp and governance thresholds are incomplete.

### Feature: AI Feedback Logging Scaffold

**Purpose**
- Intended feedback capture loop for AI quality.

**Status**
- In Progress.
```

```
**Where It Appears**
- Migration for `ai_feedback_logs` exists.
- Referenced in docs, but route/controller/service/model/test are missing.

**Primary Users**
- Intended for internal QA/ops.

**Feature Flow**
1. Unclear from code (not implemented).

**Inputs**
- Unclear from code.

**Outputs**
- Table exists but no active writes.

**Thresholds / Rules / Conditions**
- No explicit threshold found.

**Main Files**
- `database/migrations/2026_04_09_000006_create_ai_feedback_logs_table.php`

**Dependencies**
- No active runtime dependency path.

**Telemetry / Logging**
- None active.

**Risks / Caveats**
- Stakeholders may assume feedback loop is live due docs references.

**Current Gaps**
- Endpoint/service/model/tests missing.

**Presentation Summary**
- Feedback loop is not production-active.
- Treat as roadmap/in-progress, not deployed capability.

## 4. Status Summary
- Live: Triage, Memory/Tasks, Draft, Report/PDF (sync), Legacy ChatGPT.
- Partially Implemented: Omnichannel classification (channel + policy gaps), async report scaffolding.
- Experimental: none explicitly identified.
- Internal Only: telemetry/cost/dashboard usage insights.
- Disabled / Gated: runtime controlled by superadmin AI settings; settings update route gated by plan feature +
superadmin.
- In Progress: feedback endpoint stack.
- Legacy / Unclear: legacy ChatGPT architecture is active but separate from Conversation AI stack.

## 5. Thresholds and Decision Logic Summary
- Route throttle: `30/min` for Conversation AI endpoints.
- Availability gate: `ai_conversation_enabled=1` and non-empty API key.
- Timeout validation and clamp: settings `5..120`, provider min `5`.
- Triage score/state caps and transition guards as listed above.
- Auto-refresh staleness buffers: `5s` for triage/memory.
- Report scope validation and linked entity checks.
- Activity caps: `250` stream items, `20` opportunities in context snapshot.
- Draft policy gates: hard-block terminal states, body format, single-CTA, state-aware language suppression.
- Standard AI-unavailable fallback contract: `422 {"message":"AI unavailable"}`.
- Report download readiness: `409` if no result payload.
- Queue behavior: async job class exists; current report generation path is synchronous.
```

6. Architecture Overview
- Frontend:
- Conversation AI UI (drawer/cards/editor) calls `/ai/*` endpoints.
- Legacy global modal calls `/api/chatgpt/generate`.
- Routes/controllers:
- Authenticated tenant-scoped endpoints with explicit ownership checks.
- Services/jobs:
- Orchestration services + skill layers + persistence + telemetry.
- Report async job scaffold exists, currently not used in controller path.
- Prompt/context:
- Versioned prompt factories and context builder for CRM/activity enrichment.
- Provider/model:
- `OpenAiConversationClient` uses Responses API with strict JSON schema.
- Legacy ChatGPT path uses chat completions.
- Parser/formatter:
- Skill-level parse/policy validation, repair/fallback, report output normalization.
- Storage/download:
- AI tables store outputs/logs; DomPDF renders downloadable report.

7. Risks and Important Notes
- Documentation mismatch vs runtime:
- `/ai/feedback` and queue-first report flow appear in docs but not in active runtime.
- Sensitive debug logging in report path:
- Prompt and raw AI output logging may expose business context.
- Cost estimation reliability:
- model pricing map does not explicitly include configured default `gpt-5.4-mini`.
- Gating inconsistency:
- settings/UI are plan-gated; conversation endpoints primarily rely on global availability + tenancy checks.
- Legacy path sanitization:
- ChatGPT endpoint returns raw exception text in failure responses.
- Omnichannel parity:
- WhatsApp ingestion path still TODO for AI classification.

8. Suggested Presentation Slides
1. AI Capability Snapshot
- Say: two AI stacks (modern Conversation AI + legacy generator), with clear production boundaries.
2. Customer-Facing AI Journey
- Say: classify, remember, draft, export report in the Conversations workflow.
3. Decision Logic and Guardrails
- Say: strict policy/transition rules prevent unsafe or over-optimistic actions.
4. Governance and Observability
- Say: usage/failure/cost telemetry exists with executive dashboard visibility.
5. Maturity by Feature
- Say: core is live; feedback endpoint and async report path are partial/in-progress.
6. Next Hardening Priorities
- Say: activate async report worker path, implement feedback API, align gating, sanitize legacy errors.